

Introduction

Every Python developer hits this bug at least once: you write a useful function, copy it into a second file because you need it there too, then copy it again into a third file. Weeks later you find a bug in that function — and now you have to remember every place you pasted it. Miss one copy, and you've got a silent, hard-to-trace inconsistency in your codebase.

This isn't just a beginner mistake. It's the exact problem that Python's module and import system was built to solve. Understanding *how* imports actually work — not just the syntax, but what Python is doing behind the scenes — is one of those fundamentals that separates someone who writes scripts from someone who builds maintainable software. It also shows up constantly in technical interviews and code reviews: "why did you guard that with `if __name__ == '__main__':`?" is a question that trips up even experienced developers who've never had to explain it out loud.

In this lesson, we'll cover what a module actually is, the three ways to import one, how Python finds the file you're importing, why the `__main__` guard exists, and how a folder becomes a real Python package. By the end, you'll understand exactly why copy-pasting code doesn't scale — and what to do instead.

Problem Overview

Here's the situation in plain terms: you have a function, say `clean_name()`, that strips whitespace and formats a string consistently. You need this function in three different files in your project. The naive approach is to paste the function definition into each file.

This works, technically. The code runs. But it creates a maintenance trap: the three copies are only *identical* at the moment you pasted them. The instant you fix a bug or add a feature in one copy, the other two silently fall out of sync. There's no compiler error, no warning — just three functions that look the same but slowly diverge.

The real problem to solve isn't "how do I write this function" — it's "how do I write this function *once* and reuse it everywhere, so there's exactly one source of truth." That's what Python's import system is for.

Example

Imagine two files sitting in the same folder:

```
project/  
├─ greetings.py  
└─ app.py
```

`greetings.py` contains:

```
def hello(name):  
    print(f"Hello {name}")  
  
def goodbye(name):  
    print(f"Goodbye {name}")
```

`app.py` wants to use `hello()`. Instead of retyping the function, `app.py` imports it:

```
import greetings  
  
greetings.hello("Ada")
```

Running `app.py` prints:

```
Hello Ada
```

Nothing about `greetings.py` is special here — no decorator, no registration step. The moment `app.py` imports it, Python treats it as a module and gives `app.py` a reference to everything defined inside it. That's the whole mechanism: one file, one definition, borrowed wherever it's needed.

Intuition

Here's the mental model that makes this click: think of a `.py` file as a toolbox sitting in a shared garage. A **module** is just that file — its functions, variables, and classes are tools sitting on a shelf. When you `import` it, you don't copy the tools into your own toolbox. You walk over to the shared garage and use them directly, in place.

This reframes the whole problem. You're not asking "how do I get a copy of this function into my file?" You're asking "how do I point at the one function that already exists?" Once you think in terms of *pointing*, the rest of the system makes sense:

- `import x` gives you a pointer to the whole module — you reach into it with `x.something`.
- `from x import y` gives you a direct pointer to one specific name inside the module.

- `import x as z` is the same pointer, just under a different local nickname.

None of these copy code. They all just create a reference. That's also why fixing a bug in `greetings.py` fixes it *everywhere* the module is imported — there's only ever one `hello()` function in memory, no matter how many files point at it.

The second piece of intuition — the `__main__` guard — comes from a simple observation: a file behaves differently depending on how it's started. If you *run* a file directly, it's the "front door." If another file imports it, it's the "side door." Python needs a way to tell you which door was used, so you can decide whether test code, demo prints, or setup logic should actually execute. That's exactly what `__name__` is for.

Brute Force Solution

The idea: Copy the function definition into every file that needs it.

Algorithm: 1. Write the function once. 2. Paste the same code into every file that needs to use it. 3. Repeat step 2 every time you fix a bug in the function.

Advantages: - No imports to think about — the function is always "right there." - Works for a one-off script you'll never touch again.

Disadvantages: - Every bug fix has to be applied N times, once per copy. - Copies drift apart silently — no error tells you they're now different. - Doesn't scale past a couple of files; becomes unmanageable with real projects.

Complexity: - Time to fix a bug: $O(n)$ manual edits, where n is the number of copies — and that cost is paid by a human, not the computer, which is worse. - Space: $O(n)$ duplicated code stored on disk for a function that only needed to exist once.

Optimal Solution

The fix is to treat the function as living in exactly one place — the module — and have every other file *reference* it instead of duplicating it.

Step 1: Write the module once. Put `hello()` and `goodbye()` in `greetings.py`. Nothing else needs to happen for this to be a module — any `.py` file is automatically a module the moment something imports it.

Step 2: Import it where needed. Any file that needs `hello()` writes `import greetings` and calls `greetings.hello(...)`. There's one function in memory; every caller points at it.

Step 3: Guard code that shouldn't run on import. If `greetings.py` has test or demo code at the bottom, wrap it in the `__main__` guard so it only fires when the file is run directly:

```
if __name__ == "__main__":
    print(hello("test run"))
```

Here's the table that explains why this works:

How the file is started	Value of <code>__name__</code>	Does the guarded block run?
<code>python greetings.py</code> (run directly)	<code>"__main__"</code>	Yes
<code>import greetings</code> (imported from elsewhere)	<code>"greetings"</code>	No

Python sets `__name__` automatically on every module — you never assign it yourself. Run the file, and Python sets `__name__` to `"__main__"`. Import the file, and Python sets `__name__` to the module's own name. The guard is just a plain `if` statement checking that value.

Step 4: Group related modules into a package. Once you have several related files — formatting, validation, greetings — put them in a folder and add an empty `__init__.py`:

```
toolkit/
├── __init__.py
└── greetings.py
```

The presence of `__init__.py` tells Python "treat this folder as an importable package, not just a random directory." It can also control what gets exposed when someone imports the package itself, but in most real projects it's just an empty file — that's completely normal.

Now `app.py` can import through the folder:

```
from toolkit import greetings

greetings.hello("Ada")
```

Same borrowing idea as before, just with one more level of nesting.

Python Code

Here's the complete, working example, structured the way you'd actually organize it:

```
# toolkit/greetings.py

def hello(name: str) -> None:
    """Print a friendly greeting."""
    print(f"Hello {name}")

def goodbye(name: str) -> None:
    """Print a friendly farewell."""
    print(f"Goodbye {name}")

if __name__ == "__main__":
    # Only runs when this file is executed directly,
    # never when it's imported by another module.
    hello("test run")
```

```
# toolkit/__init__.py
# (intentionally empty – its presence marks this folder as a package)
```

```
# app.py

from toolkit import greetings

greetings.hello("Ada")
```

Code Walkthrough

- `def hello(name: str) -> None:` — a plain function, nothing import-specific about it. Type hints aren't required for imports to work; they're just good practice.
- `if __name__ == "__main__":` — checks the automatically-set `__name__` variable. This block is the *only* part of the file that behaves differently depending on how the file was started.
- `toolkit/__init__.py` — an empty file whose only job is to exist. Its presence is what makes `toolkit` a package that `from toolkit import greetings` can reach into.
- `from toolkit import greetings` — a dotted path: look inside the `toolkit` package, find the `greetings` module, and bind the name `greetings` in `app.py`. Note this imports the *module*, not a specific function, so you still call `greetings.hello(...)`.
- `greetings.hello("Ada")` — reaches through the module reference to call the function. Because it's a reference, not a copy, this always calls the current version of `hello()`, wherever it was last edited.

Dry Run

Let's trace exactly what happens when you run `python app.py` :

1. Python starts executing `app.py` . `app.py` 's own `__name__` is set to `"__main__"` .
2. Python hits `from toolkit import greetings` . It has never seen `toolkit` before, so it locates the `toolkit/` folder, sees `__init__.py` , and confirms it's a valid package.
3. Python locates `greetings.py` inside `toolkit/` , and executes it top to bottom — this is the "import runs the file once" step. During this execution, `greetings.py` 's `__name__` is set to `"toolkit.greetings"` (not `"__main__"` , because it was *imported*, not run directly).
4. Because `__name__` inside `greetings.py` isn't `"__main__"` , the guarded `hello("test run")` call is skipped entirely.
5. Control returns to `app.py` . The name `greetings` now points at the fully-executed module.
6. `app.py` calls `greetings.hello("Ada")` , which runs the function body and prints `Hello Ada` .

Now compare: if you instead ran `python toolkit/greetings.py` directly, step 3 never happens as an import — `greetings.py` itself becomes the entry point, `__name__` is set to `"__main__"` , and this time the guard *does* let `hello("test run")` execute, printing `Hello test run` .

Complexity Analysis

This isn't a classic algorithmic problem, but it's worth reasoning about cost the same way:

- **Time:** A module is executed exactly once per process, no matter how many times it's imported elsewhere — Python caches it in `sys.modules` after the first import. So importing the same module from five different files costs one execution, not five. Subsequent imports are $O(1)$ dictionary lookups.
- **Space:** One copy of the function lives in memory, referenced by every importer — $O(1)$ relative to the number of files that use it, compared to the brute-force $O(n)$ copies sitting on disk.
- **Why this matters for correctness:** because there's exactly one execution and one in-memory copy, a bug fix applied once is visible everywhere immediately. There's no synchronization problem because there was never more than one copy to begin with.

Alternative Solutions

Relative imports (`from . import greetings`) inside a package work similarly to `from toolkit import greetings` , but are written relative to the current package rather than by full name. They're

useful inside larger packages but can get confusing when a file is sometimes run directly and sometimes imported — a common source of `ImportError: attempted relative import with no known parent package`. For a project this size, absolute imports (`from toolkit import greetings`) are simpler and less error-prone.

Adding the folder to `sys.path` manually and importing without a package structure is another option, but it bypasses the safety of `__init__.py`-based packaging and tends to cause the exact naming collisions described below. Prefer proper packages unless you have a specific reason not to.

Edge Cases

- **Empty file imported as a module:** perfectly legal — you just get a module with no names in it. Common for placeholder files during early development.
- **Circular imports:** if `a.py` imports `b.py` and `b.py` imports `a.py`, Python will raise an error or return a partially-initialized module, because each file tries to execute before the other has finished. Avoid by restructuring shared code into a third module both files import from.
- **Name shadowing a standard library module:** naming your file `random.py` in a folder that's on `sys.path` will cause `import random` to load *your* file instead of the standard library — and any code expecting `random.randint()` will break with a confusing error.
- **Missing `__init__.py`:** in most modern Python versions this still often works via "namespace packages," but relying on that is fragile and surprising — always add the file explicitly for a real package.
- **Importing a module that has top-level code with side effects** (like a `print()` outside any guard): that code runs on import, whether you wanted it to or not — this is precisely what the `__main__` guard prevents.

Common Mistakes

1. **Forgetting the `__main__` guard entirely**, so test/demo code fires every time the module is imported anywhere.
2. **Naming a file after a standard library or third-party module** (`random.py`, `json.py`), silently shadowing it.
3. **Confusing `import x` with `from x import y`** and then calling the function without the module prefix, e.g. writing `hello()` instead of `greetings.hello()` after `import greetings`.
4. **Forgetting `__init__.py`** and being confused why the folder "isn't a package" in older setups or certain tooling.

5. **Copy-pasting functions between files** instead of importing, recreating the exact bug this lesson exists to fix.
6. **Assuming import "runs the code every time it's called"** — it only executes the module body on the *first* import; later imports reuse the cached module.

Interview Questions

- What's the difference between a module and a package in Python?
- What does `__name__` equal when a file is run directly versus imported? Why does Python design it this way?
- Walk through what happens, step by step, when `import mymodule` executes.
- What's the difference between `import x`, `from x import y`, and `import x as z` — when would you choose each?
- What is `sys.path`, and how does Python decide where to look for a module?
- What problem does `__init__.py` solve, and what happens if you import a folder without one?
- Why can a locally named file "break" a standard library import? How would you debug that?

Similar Problems

- **Design a plugin/registration system in Python** — builds directly on understanding how imports execute module-level code exactly once.
- **Refactor a script into a reusable library** — the practical, real-world version of "stop copy-pasting functions," directly reflecting this lesson.
- **Debug a circular import error** — tests whether you understand *when* module code actually executes, not just import syntax.
- **Implement a namespace/relative import structure for a multi-package project** — extends the package concept covered here to larger codebases.

Key Takeaways

- A module is just a `.py` file — nothing more mystical than that.
- Import doesn't copy code; it runs the file once and hands you a reference, so there's always exactly one source of truth.
- `import x`, `from x import y`, and `import x as z` all do the same job with different amounts of typing — pick for readability, not brevity.
- Every file has a `__name__` variable that's `"__main__"` when run directly and the module's own name when imported — that's the entire mechanism behind the `if __name__ ==`


```
"__main__": guard.
```

- A package is just a folder with an `__init__.py`, letting you group related modules under one importable name.

Watch the Video

This lesson is easier to absorb watching it built step by step — {LINK} walks through writing `greetings.py`, importing it two different ways, and watching the `__main__` guard fire (or not) in real time.

About the Series

This lesson is part of *Fun with Learning Technology*, a series that breaks down Python and software engineering fundamentals — the kind of concepts that show up constantly in real code and in interviews but rarely get explained from first principles. Each lesson builds a small, working example so you can see the concept in action, not just read about it.

Call To Action

If this cleared something up, give the video a like on YouTube — it helps the series reach more developers working through the same fundamentals. Subscribe on YouTube for the rest of the series, and subscribe to the Substack newsletter for write-ups like this one delivered straight to your inbox. Drop a comment with what confused you most about imports before this, and share it with anyone still copy-pasting functions between files.

Quick Review

Bug: Python found the wrong module. Likely cause?

Name collision — a local file/folder shares a name with a stdlib or third-party module and shadows it on the import path.

What is a 'module' in Python, fundamentally?

Just a .py file — its top-level names become attributes accessible via import.

Three ways to import — name them.

`import module`; `from module import name`; `import module as alias` (aliasing).

Where does Python search for modules, in order?

Current script's dir, then `PYTHONPATH` dirs, then `stdlib`, then site-packages (`sys.path` order).

Why does `if __name__ == '__main__':` matter?

Lets a file behave as a library (importable, no side effects) or a script (runs main logic) depending on how it's invoked.

What file marks a directory as a package?

`__init__.py` (even empty) — without it, imports across sibling modules can fail or behave inconsistently.

Python Lesson 16: Modules and Packages (import, creating your own modules, `__name__ == '__main__'`, and package structure) — free Python cheat sheet